

Building a Production Retrieval-Augmented Generation System

Architecture, Implementation, and Evaluation of a Fully Local RAG Pipeline

Prepared by Roy Abi Nader

InMind Academy

Sumer 2026

Table of Contents

Table of Contents.....	2
1. Introduction.....	5
2. System Architecture Overview	5
2.1 pipeline.py.....	6
2.2 stages.py.....	6
2.3 load_step.py	6
2.4 api.py.....	6
3. Ingestion.....	6
3.1 Files in this stage.....	7
3.2 The underlying problem: Archive.org scans.....	7
3.3 Difficulty 1: distinguishing real illustrations from scan artifacts	7
3.4 Difficulty 2: pages with no text layer at all.....	8
3.5 Difficulty 3: image context (unverified)	9
3.6 Why this ingestion method was chosen	9
3.7 Current status	10
4. Chunking.....	11
4.1 Methods implemented.....	11
4.2 Difficulties encountered.....	11
4.3 Results on the real corpus	12
5. Embeddings.....	13
5.1 Backends and models implemented.....	13
5.2 Difficulties encountered.....	13
5.3 Results on the real corpus	15
5.4 Multimodal image handling: captions embedded alongside text.....	15
6. Vector Storage	17
6.1 Results on the real corpus	17
7. Retrieval.....	18
7.1 Methods implemented.....	18
7.2 Difficulties encountered.....	18
7.3 Results on the real corpus	20
8. Generation.....	20
8.1 Backends and models implemented.....	20

8.2 Results on the real corpus	21
9. Safety	22
10. Evaluation	23
10.1 Tools implemented.....	23
10.2 Difficulties encountered.....	23
11. Additional Exploration.....	25
11.1 Small language models	25
11.2 Quantization.....	25
11.3 Vision language models.....	25
11.4 Serving infrastructure.....	25
12. Utilities.....	27
13. Running the System	28
14. Evaluation Metrics Explained.....	29
14.1 Ingestion.....	29
14.2 Chunking.....	29
14.3 Embedding	29
14.4 Storage	29
14.5 Retrieval.....	29
14.6 Generation.....	30
14.7 A more detailed alternative	30
15. Conclusion	30
Appendix A: Development History	32
Phase 1: Initial review and first round of fixes	32
Phase 2: Extending method comparison to new code.....	32
Phase 3: Building the checkpointed comparison tool	32
Phase 4: Bugs found only once real data was used.....	32
Phase 5: Automated relevance labeling	32
Phase 6: Evaluation set hygiene.....	33
Phase 7: Documentation	33
Phase 8: Activating an unused prompt injection defense	33
Phase 9: Incremental ingestion silently duplicating image vectors	33
Phase 10: A first approach to multimodal retrieval, through captioned images and CLIP's text tower	33
Phase 11: A circuit breaker for a silently broken captioning backend	34

Phase 12: Import time cost mistaken for a runtime problem	34
Phase 13: A system proxy silently discarding image requests to a local model server	35
Phase 14: A second, independent branch approach to multimodal retrieval	35
Phase 15: Defects found while building the second approach.....	35
Phase 16: Identifiers that did not survive re-ingestion.....	36
Phase 17: Repeated model loading inside a long running server.....	36
Phase 18: A metadata value the vector store's own validation rejected.....	36
Phase 19: A single unreadable image ending an otherwise successful run	37
Appendix B: Model and Method Reference Table	38
B.1 Embedding models.....	38
B.2 Generation models.....	38
B.3 Vision language models	39
B.4 Reranking model	39
Appendix C: Production Deployment on the Real Corpus.....	40
C.1 Per-file ingestion results.....	40
C.2 Chunking and embedding at production scale	41
C.3 Storage.....	41
C.4 Generation model completion status	41
C.5 Current deployment status.....	42

1. Introduction

This report documents the design, implementation, and evaluation of a local retrieval-augmented generation (RAG) system. The system answers questions over a private document collection by retrieving relevant passages and using a language model to generate a grounded answer from them, instead of relying on a model's built-in knowledge alone.

Every model and tool used in this project runs locally and is free to use. No paid API keys are required anywhere in the pipeline. Language models and embedding models are served either through a local Ollama server or through Hugging Face model weights downloaded once and cached on disk. This principle is stated here once and applies throughout the rest of this report.

The document collection used to build and test this system consists of historical painting and drawing treatises scanned from Archive.org. These documents present real practical challenges typical of scanned archival material, including pages with no extractable text layer, embedded background scan images, and optical character recognition (OCR) noise. Several sections of this report describe how these challenges were identified and addressed.

The pipeline is organized into six stages: ingestion, chunking, embedding, storage, retrieval, and generation. Each stage is implemented with several interchangeable methods, so that different approaches can be compared on real data rather than chosen by assumption. This report explains what each stage does, which methods were implemented, what difficulties were encountered while building and testing each one, and what the resulting evaluation numbers show.

The report is organized as follows. Section 2 gives an overview of the system architecture and the tools used to run it. Sections 3 through 10 describe each pipeline stage in turn. Sections 11 and 12 describe supporting and exploratory components. Section 13 explains how the system is actually run. Section 14 explains the metrics used to evaluate each stage. Section 15 concludes with a summary of results and limitations. Appendix A records the development history of the project in chronological order, and Appendix B provides a consolidated reference table of every model used.

2. System Architecture Overview

The pipeline processes document collection through six sequential stages.

1. Ingestion: raw files (text, PDF, or image) are parsed into a common internal format called a RawDocument.
2. Chunking: each document is split into smaller pieces of text suitable for retrieval.
3. Embedding: each chunk of text (or image) is converted into a numeric vector that represents its meaning.
4. Storage: vectors are written to a vector database that supports fast similarity search.
5. Retrieval: given a question, the system finds the chunks most likely to answer it.

6. Generation: a language model produces an answer using the retrieved chunks as context.

The pipeline can be run in four different ways, all of which use the same underlying code for each stage.

2.1 pipeline.py

The original command line interface. The ingest command performs stages 1 through 4 in one step, and the ask command performs stages 5 and 6 in one step. This tool always writes to a single fixed vector store collection named rag_chunks.

2.2 stages.py

Splits the six stages into independent commands. Each command reads its input from a checkpoint file written by the previous command and writes its own checkpoint file, so any stage can be re-run without repeating the stages before it. Each run of a stage still uses exactly one method, chosen by command line flag. This tool also writes to the same fixed rag_chunks collection as pipeline.py.

2.3 load_step.py

Also splits the pipeline into six independent, checkpointed stages, but instead of running one method per stage, each stage runs every applicable method and records comparison metrics for all of them. Results are written to vector store collections named after the specific combination of chunking method and embedding model being tested, so that experiments never overwrite the production collection or each other. This tool is the primary instrument used to compare methods and choose a configuration for the rest of this report.

2.4 api.py

A REST API built with FastAPI, exposing an ingestion endpoint, a query endpoint, a query comparison endpoint, a document listing endpoint, and a health check endpoint. It reuses the same underlying functions as pipeline.py and reads from the same rag_chunks collection.

The relationship between these tools is intentional. load_step.py is used to compare methods and decide on a configuration using real, measured evidence. stages.py, pipeline.py, and api.py are then used to build the production system using the configuration that the comparison work justified. Nothing transfers automatically between the two groups; the chosen configuration must be re-run through stages.py or pipeline.py to become the live system.

3. Ingestion

The ingestion stage takes raw files sitting on disk, such as text files, PDFs, and images, and converts them into a common internal structure called a RawDocument, made up of extracted text (or an image path) plus a metadata dictionary recording the source filename, page number,

and how the text was obtained. This is the only stage that reads files directly; no chunking, embedding, or retrieval logic exists in this folder.

3.1 Files in this stage

File	Purpose
schema.py	Defines the RawDocument and Chunk data structures used throughout the rest of the pipeline.
ingest_text.py	Reads plain text and Markdown files.
ingest_pdf.py	Reads PDF files: extracts the text layer page by page, extracts embedded images, and falls back to OCR on pages with no text layer.
ingest_image.py	Reads standalone image files.
loader.py	Dispatches each file in a source folder to the correct reader above and returns one combined list.
table_extraction.py	Extracts tables from PDFs as structured markdown using pdfplumber, instead of leaving them as flattened paragraph text.
incremental_indexer.py	Tracks a content hash per source file so re-running ingestion only processes new or changed files.
deduplication.py	Removes exact and near duplicate chunks before they are embedded and stored.
cache.py	Saves and loads the ingestion output to disk, so the slowest step in the pipeline only needs to run once per document set.

3.2 The underlying problem: Archive.org scans

The document collection used in this project consists of historical painting and drawing treatises pulled from Archive.org. This detail matters because Archive.org PDFs are not ordinary, digitally created PDFs. A normal PDF has real, selectable text, and any images inside it are deliberate illustrations placed by the author. Archive.org files are scans of physical books, which introduces two separate problems that required two separate solutions.

3.3 Difficulty 1: distinguishing real illustrations from scan artifacts

The first version of the PDF ingester extracted every embedded image on every page, on the assumption that these could be genuine figures worth keeping for a later multimodal retrieval step. Running this against the real corpus produced a result such as 6 text files, 6,515 images, and 1,627 PDF text pages. A ratio of roughly four images per page is not a sign of real illustrations; it is a sign of the scanning process itself.

Most Archive.org PDFs work as follows: the visible page is a full page photograph of the physical page, with an invisible layer of OCR text placed over it so the page can be searched. That full page photograph is stored in the PDF as an embedded image, so a naive image

extractor pulls out one image per page, and often more than one, since scanning software commonly also saves a transparency mask and a distorted preview thumbnail as separate image objects.

The first attempted fix checked each image's pixel width and height and compared its aspect ratio to the page's aspect ratio. Images that were roughly page shaped and larger than about 800 by 800 pixels were treated as background scans and discarded. This reduced the image count considerably, but two specific failure cases remained, both found by inspecting individual problem pages by hand.

- Some background scans were shipped at a lower resolution than 800 pixels on a side, since Archive.org sometimes provides a smaller reading copy scan rather than the full archival resolution version. These slipped through the pixel size check untouched.
- Archive.org's own BookReader software embeds a second image per page: a deliberately warped thumbnail used for its on screen page turn animation. Because it is warped, its pixel aspect ratio does not resemble a normal page, so the aspect ratio check incorrectly treated it as a genuine figure.

The effective fix was to stop checking an image's own pixel dimensions and instead check where the image is physically placed on the page. PyMuPDF can report the rectangle into which a given image is drawn, regardless of the image's own resolution or whether it has been stretched or skewed to fit that rectangle. If that rectangle covers most of the page area, the image is treated as a background scan, regardless of its raw pixel content. This single change correctly caught both the low resolution scans and the warped thumbnails, since both are still placed to cover nearly the entire page even though their pixel content is unusual.

This logic was verified in two ways, since no real Archive.org files were available in the environment used to build and test it. First, synthetic PDFs were constructed from scratch, containing a fake page with a low resolution background image, a small genuine figure, and a distorted image placement, to confirm the whole pipeline still worked end to end. Second, a direct unit test fed the filtering function a set of hand constructed geometry cases, including a warped thumbnail whose bounding box covers ninety five percent of the page, a genuine small figure, a low resolution full page scan, and a case where no placement data exists at all.

3.4 Difficulty 2: pages with no text layer at all

The image filtering problem above assumes a text layer already exists on the page. This is true for most of the corpus but not all of it; a small number of books turned out to be almost entirely raw scans with no text layer whatsoever. On these pages the words are simply part of the photograph, since nothing was ever recognized as text for that page.

The solution is straightforward: if a page's native text comes back empty, or nearly empty (a stray page number does not count as real text), the page is rendered to an image and processed with local OCR using pytesseract, and the OCR output is used as that page's text instead.

These two problems interact without conflicting. A page can have a full native text layer and still contain a background scan image that needs filtering, which is the typical Archive.org case. A page can also have no native text at all, in which case OCR fills the gap while the same placement based filter still correctly removes the underlying scan image, since that filter does not depend on whether the text was obtained by extraction or by OCR.

One practical complication is worth recording. The Tesseract OCR binary must be discoverable on the operating system, which is not guaranteed simply by installing the Python wrapper. This is a particularly common problem on Windows, where Tesseract can be installed correctly and still be invisible to Python because it is not on the system PATH. A configurable override was added so the exact installation path can be specified directly rather than relying on automatic detection.

3.5 Difficulty 3: image context (unverified)

A further improvement was described in project notes: once genuine figures are correctly kept rather than lost among thousands of scanned backgrounds, each kept image on its own is still just a picture with a filename, with no indication of what page it illustrates or what the surrounding text says about it. The described approach was to locate text blocks near an image's position on the page and attach that text to the image as metadata.

3.6 Why this ingestion method was chosen

The assignment requirements allow native text parsing, OCR, or a vision language model as the method for handling difficult scanned documents, and require the choice to be justified. This project uses native extraction first, with OCR used only as a fallback when native extraction returns nothing for a page. It deliberately does not run OCR unconditionally on every page, and does not use a vision language model as the primary ingestion method.

Running OCR on every page regardless of whether a native text layer already exists would waste computation. Archive.org scans typically already carry a native text layer for a large share of pages, and native extraction on such a page is effectively free, since it involves reading embedded text rather than running a model. Running OCR anyway would cost real time for a result that is strictly worse, since OCR introduces recognition errors that a proper native layer does not have. Other stages in this same pipeline proved expensive at real corpus scale, including one chunking method that took over one thousand seconds and one embedding model that took over an hour on the full corpus, which reinforces the importance of avoiding unnecessary computation wherever native extraction already succeeds.

A vision language model was not chosen as the primary method because it is the wrong tool for high fidelity, verbatim text transcription at this scale. It is slower per page than either native extraction or OCR, and vision language models are built for visual reasoning about an image rather than for reliably transcribing dense running prose word for word across thousands of pages. This project does use a vision language model, described in Section 11, but for a task it is actually suited to: reasoning about one already retrieved image at generation time, not transcribing an entire corpus.

Using OCR only as a fallback, rather than skipping it entirely, satisfies the requirement that a scanned page must never be silently returned as empty text. A native only approach would fail on any page without a text layer. The fallback closes that gap directly, since native extraction is attempted first and OCR only runs when it is needed.

There is direct evidence that this fallback path is doing real work rather than sitting unused. Two separate downstream symptoms trace back to genuinely OCR processed pages in this corpus. One chunking method produced a single chunk of 924 words that the sentence splitting logic treated as one sentence, which happens when OCR loses normal punctuation and leaves the sentence tokenizer nothing to split on. Another chunking method, which splits text based on topic similarity, produced unusually small and inconsistent chunks, which is consistent with noisy OCR text producing noisy sentence embeddings.

The method is not without limitations. One chunking method produced a single chunk of 85,938 words from what should have been ordinary page sized content, which points to at least one page where extraction merged more content than intended, possibly an index or table of contents, or an extraction error spanning more than one page. This is a real limitation worth stating directly: the overall ingestion approach is sound, but this specific corpus surfaced at least one edge case that the current logic does not catch. A maximum page length sanity check on extraction output is a reasonable follow up before considering ingestion fully hardened.

A final deliberate design choice is worth noting. Embedded images are extracted from PDF pages as their own separate documents rather than left inside an opaque page object, and full page background scans are filtered out specifically when real text already exists underneath them. This separation is what makes image based retrieval possible at all, since an illustration only becomes independently searchable once it exists as its own unit, and the background scan filter avoids spending embedding computation on page sized images that carry no distinct illustrative content.

3.7 Current status

In its current form, the PDF ingester processes a document page by page. For each page, it first attempts native text extraction, and uses that text if enough of it is present. If not enough native text is present, it renders the page and runs OCR instead. It then examines every embedded image on the page and checks how much of the page area it is placed to cover; images covering most of the page are treated as background scans or animation thumbnails and dropped. Every module in this stage runs on free, local tools, specifically PyMuPDF for PDF parsing and pytesseract for OCR, with no paid services involved anywhere.

4. Chunking

By the time a document reaches the chunking stage, it has already been converted into raw text with metadata. A raw document can still represent an entire page or an entire book, which is too large to embed and retrieve precisely. The chunking stage divides that text into smaller pieces, large enough to contain a complete idea but small enough to embed and retrieve accurately.

4.1 Methods implemented

Method	Description
fixed_size	Cuts text every 500 words with 100 words of overlap, without regard for sentence or paragraph boundaries.
recursive	Uses the same target size as fixed_size, but tries paragraph breaks first, then sentence breaks, then word breaks, only falling back to a cruder cut when nothing better fits.
sentence_based	Groups every four whole sentences, found using NLTK's sentence tokenizer, into one chunk.
semantic	Embeds every sentence and starts a new chunk wherever the similarity between consecutive sentences drops below a threshold. Requires an embedding model to be supplied.
structure_aware	Splits on markdown headings for plain text, or produces one chunk per page for already page segmented PDF text.
parent_child	Builds larger 800 word parent chunks first, then splits each parent into smaller 200 word child chunks. Only the children are embedded and searched; the corresponding parent is substituted in at generation time to provide fuller context.

4.2 Difficulties encountered

Testing across all six methods against the real corpus surfaced seven distinct issues, described here in the order they were found.

An extraction outlier

Running the structure_aware method against the real corpus produced one chunk of 85,938 words, compared to an average chunk size of 395 words for that method. The standard deviation across all chunks from this method was roughly six times the average, which is the statistical signature of one extreme outlier. This points to a page extraction problem rather than a flaw in the chunking rule itself, since one page to one chunk is a reasonable rule when the underlying page is actually page sized.

The cost and inconsistency of semantic chunking

The semantic method requires one embedding call per sentence to decide where to place chunk boundaries. Against the real corpus of roughly 3,674 pages, this took 809 seconds in one run and 1,033 seconds in a repeated run against the same unchanged data, a difference of about 28

percent with nothing about the input changed. This method also requires an embedding model to be loaded before it can run at all, unlike the other five methods, which need only the raw text.

Over fragmentation from semantic chunking

The semantic method produced an average chunk size of only 19 words, with the highest relative variability of any method tested. This is consistent with the similarity threshold being tripped on nearly every sentence, which in turn is consistent with noisy OCR text producing inconsistent sentence embeddings that do not cluster the way clean prose would.

Sentence tokenization errors from OCR text

NLTK's sentence tokenizer relies mainly on punctuation to detect sentence boundaries. Pages that lost their punctuation during OCR give it nothing to split on, so an entire garbled block of text is treated as one sentence. This produced a single 924 word sentence in the real corpus, even though the sentence_based method's average chunk size is only 70 words.

A hidden external dependency

Both sentence_based and semantic chunking depend on NLTK's sentence tokenizer data files, which are not bundled with NLTK itself and must be downloaded once before either method can run. If that download has not occurred, both methods fail immediately, which can appear to be a code defect rather than a missing setup step.

Comparing six methods with very different output shapes

Running all six methods on identical input produced chunk counts ranging from 3,674 to 76,022 for the same source text, a twenty fold spread. Chunk count alone says little about quality when the shapes differ this much. A fair comparison required looking at average size, minimum, maximum, and standard deviation together, and specifically the standard deviation relative to the average, since that measure is what actually revealed both the structure_aware outlier and the semantic fragmentation problem.

4.3 Results on the real corpus

The following table summarizes the six chunking methods measured against the real document collection, consisting of 3,668 PDF text pages, 1,301 embedded images, and 6 plain text files.

Method	Chunks	Avg words	Max words	Std dev	Time (s)
fixed_size	4,743	328.4	500	145.1	0.26
recursive	6,353	246.3	555	109.7	0.31
sentence_based	20,686	70.1	924	54.1	2.32
semantic	76,022	19.1	560	22.4	809 to 1,033
structure_aware	3,674	394.8	85,938	2,461.6	0.07

Method	Chunks	Avg words	Max words	Std dev	Time (s)
parent_child	15,314	101.3	292	42.7	0.79

Table 4.1. Chunking method comparison on the real document collection.

Two methods were ruled out directly from this data. The `structure_aware` method contains a genuine extraction bug rather than a chunking strategy problem, evidenced by its single 85,938 word chunk, and should not be used until the underlying page extraction issue is fixed. The semantic method is not practical at this scale, taking over 800 seconds to chunk while producing severely fragmented output.

Between the remaining methods, recursive is a reasonable single method choice: bounded, fast, consistent, and more respectful of natural text boundaries than `fixed_size` for the same computational cost. The `parent_child` method achieved the best consistency of any method tested, with the lowest relative standard deviation of all six, and is the only method that provides both precise retrieval chunks and full context chunks for generation at the same time. Given that this corpus consists of long form historical prose rather than short factual snippets, this property carries real practical weight, and `parent_child` was selected as the chunking method used for the remainder of this project.

5. Embeddings

Chunking produces pieces of text. None of that text is directly searchable, since a computer cannot compare meaning directly. The embedding stage converts each chunk of text, or each image, into a fixed length list of numbers positioned so that chunks with similar meaning end up close together in that numeric space. Once every chunk is represented this way, finding the passage relevant to a question becomes a distance calculation that a vector database can perform quickly across thousands of chunks.

5.1 Backends and models implemented

Backend	Description	Models
Hugging Face	Runs directly inside the same process using the sentence-transformers library. Model weights are downloaded once and cached locally. All texts in a batch are encoded in a single call.	all-MiniLM-L6-v2 (384 dimensions), bge-small-en-v1.5 (384 dimensions), bge-base-en-v1.5 (768 dimensions)
Ollama	Sends a network request to a separately running Ollama server for each piece of text, with no batching.	nomic-embed-text (768 dimensions), mxbai-embed-large (1024 dimensions)
CLIP (open_clip)	The only backend that also embeds images, in the same vector space as text, with no captioning step involved.	ViT-B-32 with pretrained weights laion2b_s34b_b79k (512 dimensions)

5.2 Difficulties encountered

Two different serving architectures behind one interface

The Hugging Face and CLIP backends load model weights directly into the running process. The Ollama backend does neither; it sends network requests to a separate program that must already be running and have its models downloaded ahead of time. Every embedder in this project shares one common interface, so the same line of calling code has to behave correctly whether it is about to run a neural network directly on the local machine or send a request to a program that might not even be running. Every failure mode, whether a model is not downloaded, a server is not running, or a model name is incorrect, had to surface as the same kind of error regardless of which backend caused it.

The Ollama backend's hidden cost: no batching

The Hugging Face backend sends a whole list of texts to the model in one call. The Ollama backend, as implemented, sends one network request per individual chunk in a loop. This difference is not visible from the method signature, since both simply take a list of texts and return a list of vectors, but it shows up directly in measured timing. On the same real corpus, the fastest Hugging Face model averaged 70 milliseconds per chunk, while the Ollama model nomic-embed-text averaged 271 milliseconds per chunk, nearly four times slower, a gap coming largely from repeated network overhead rather than the underlying model size.

A model that failed partway through and lost all progress

Testing mxbai-embed-large against the real corpus produced the error that the input length exceeds the context length. At least one chunk in the corpus produced more tokens than this model accepts in a single call, plausibly because the corpus's OCR derived text can tokenize far less efficiently than its raw word count suggests. This also revealed that the embedding stage wraps error handling around an entire model's full run rather than around each individual chunk, so hitting one oversized chunk anywhere in a run of thousands aborted the entire attempt for that model, discarding whatever progress had already been made.

A design tradeoff that resembles a bug until it is understood

Models such as bge and nomic-embed-text are trained to be used with a text prefix that differs depending on whether the text being embedded is a search query or a document to be searched. This project deliberately does not add those prefixes, so that every embedding backend can be called with an identical plain text list signature and no per model special casing. The consequence is that these particular models do not show their best possible performance in a simple sanity check that compares a paraphrase pair against an unrelated pair. On the real corpus, all-MiniLM-L6-v2, which has no such prefix requirement, produced a much cleaner separation between similar and unrelated text than bge or nomic-embed-text did, even though those models are larger and, in nomic's case, use more output dimensions.

Why every model is checked on two fixed sentences first

Before any real corpus text is embedded, each candidate model is first tested against one paraphrase pair and one unrelated pair. This is not a test of the corpus; it is a test of whether the model is functioning correctly at all. The value of this check is concrete: nomic-embed-text

took over one hour to embed the full real corpus. If that model were misconfigured, the only way to discover it without this check would be to wait the full run and then notice poor retrieval quality afterward, with no easy way to isolate the cause. The two sentence check catches this kind of failure in under a second, before the expensive part even begins.

Determining vector size before any vector exists

The vector database needs to know how many numbers are in each vector before it can be configured to store any of them. Some libraries can report this only after the entire model has already been loaded, and the Ollama client library does not expose it at all in advance. The practical solution was a small lookup table mapping each known model name to its known output dimension, rather than depending on every library to expose this value consistently.

5.3 Results on the real corpus

The following table summarizes embedding results for five candidate models, run on the 15,314 parent_child chunks from the real corpus. The gap column is the difference between the similarity score on a paraphrase pair and the similarity score on an unrelated pair, used as a measure of how cleanly a model separates similar from different meaning.

Model	Dimensions	ms/chunk	Gap	Sane
all-MiniLM-L6-v2	384	94.0	0.746	Yes
bge-small-en-v1.5	384	70.3	0.545	Yes
bge-base-en-v1.5	768	217.5	0.547	Yes
nomic-embed-text	768	270.6	0.546	Yes
mxbai-embed-large	n/a	n/a (failed)	n/a	n/a

Table 5.1. Embedding model comparison on the real document collection.

all-MiniLM-L6-v2 was the strongest candidate from this data: nearly the fastest of the working models, clearly the best separation between similar and unrelated text, and the smallest, cheapest output size among the top performers. bge-base-en-v1.5 and nomic-embed-text both cost three to four times more processing time for a lower separation score than the cheaper options, with nothing in this data justifying their additional cost. all-MiniLM-L6-v2 was therefore selected as the embedding model used for the remainder of this project. This result should be read as a proxy on one fixed sentence pair rather than a direct measurement of retrieval quality; Section 7 presents the corresponding retrieval results on labeled real questions.

5.4 Multimodal image handling: captions embedded alongside text

Beyond the plain image embedding path described above, each image extracted during ingestion is also described by a small vision language model, which produces a short caption in ordinary language. That caption, not the image's raw pixels, is what gets embedded, through the same text embedding model used for every ordinary chunk rather than through CLIP, and

it is stored in the same collection as the rest of the document's text; a plain question can therefore retrieve an image purely from the meaning of its caption, exactly as it would retrieve any other passage. The image itself is also embedded a second time, through CLIP, into its own collection kept separate from the text collection, so an image remains reachable by direct visual similarity to a question even in cases where its caption fails to capture the specific detail being asked about. A single question is checked against both paths at once, the caption based text path and the direct visual path, and an image can be returned by either one independent of the other.

6. Vector Storage

The storage stage holds every vector together with its text and metadata, and answers similarity search requests quickly regardless of corpus size. Two backends were implemented and compared.

Backend	Description
Chroma	Embedded, requiring no separate server, and persisting to local files. Suitable for prototyping and single machine use.
Qdrant	A separate server process, written in Rust, built for production and concurrent use.

6.1 Results on the real corpus

Both backends were tested with the same 15,314 real vectors from the parent_child chunking method.

Metric	Chroma	Qdrant
Upsert time (total)	25.4 s	11.3 s
Average query time	2.4 ms	21.56 ms
Self match accuracy	100%	100%

Table 6.1. Vector store comparison on the real document collection.

Qdrant writes data roughly twice as fast, which reflects its purpose built bulk insert engine. Chroma answers queries roughly nine times faster, since its queries run inside the same process with no network call involved, while Qdrant's queries travel over the network to a separate server even on the same machine. For a single user system where queries happen far more often than writes, query speed matters more than write speed, so Chroma was selected for the production system used in this project. Both backends achieved perfect self match accuracy, confirming that vectors and their identifiers remained correctly aligned through the storage process.

7. Retrieval

Given a real question, the retrieval stage finds the small number of chunks, out of possibly tens of thousands, that are likely to answer it. This is the step that determines whether the system can actually answer questions well, since the generation stage can only work with whatever retrieval provides it.

7.1 Methods implemented

Method	Description
vector	Embeds the question and finds the closest chunk vectors by cosine similarity.
hybrid	Runs both vector search and BM25 keyword search, then merges the two ranked lists using Reciprocal Rank Fusion.
reranker	A cross encoder second pass that scores the question and each candidate chunk together, more accurate but slower, used to refine an initial set of candidates.
multi_query	Uses a language model to generate several paraphrased versions of the question, retrieves for each, and fuses the results.
query_router	A rule based classifier that selects vector search, metadata filtering, or keyword heavy hybrid search depending on the shape of the question.
contextual_compression	Uses a language model to trim each retrieved chunk down to only its sentences relevant to the question, before generation.

7.2 Difficulties encountered

Hybrid search silently dropping source information

When the hybrid method merges vector and keyword ranked lists, the merged output was carrying forward only chunk identifier, text, and score, and not the filename or page metadata. This meant every hybrid retrieved result had already lost its source attribution before generation, a separate occurrence of a similar problem already found and fixed in the chunking stage. This was corrected by tracking each chunk's metadata alongside its fused score during the merge.

Hybrid search rebuilding its index on every question

BM25 keyword search requires the entire text corpus to be available before it can rank anything, so it cannot be queried one item at a time the way a vector store can. As implemented, every single call to hybrid search fetches the entire indexed corpus and rebuilds this keyword index from scratch before comparing anything. Against a small test set this cost is invisible, but against the real corpus of over fifteen thousand chunks, asked dozens of real questions, this made hybrid search roughly one hundred times slower per question than plain vector search, confirmed directly by measured timing on real data.

A fully implemented method that was not reachable

Hybrid search existed as complete, working code, but no command line option or API parameter actually allowed selecting it; requesting it produced an explicit not implemented error. The underlying logic was correct on its own, and the gap was only found by actually attempting to use every documented feature rather than by reading the code.

The query router crashing on its own special case

The router's keyword heavy route depends on the same whole corpus keyword index that hybrid search needs, but nothing was supplying that corpus by default, so a question correctly classified into that route would then immediately fail trying to obtain it. This was corrected so the router fetches the corpus only when a question is actually classified into the route that needs it, avoiding the same cost problem hybrid search has on every question.

The reranker reloading its model on every request

The cross encoder reranker is a neural network that must be loaded into memory before use. As originally connected to the live API, a new instance was created from scratch on every incoming request that asked for reranking, repeatedly paying the model loading cost. This was corrected by loading the model once and reusing the same instance across requests, the same fix already applied to the language model used for generation.

More advanced strategies performing worse on real data

Hybrid search and multi-query expansion were both expected to outperform plain vector search, which is the common result reported for these techniques generally. On this project's real, labeled questions, both scored worse than plain vector search on precision and recall, consistently across two separate rounds of real questions. The likely explanation for hybrid search is that this corpus's real questions are mostly abstract and conceptual, which plain keyword matching handles poorly, so blending in a keyword based ranking added noise rather than useful signal, and the fusion method has no way to know in advance which of its two inputs is weaker for a given question. The likely explanation for multi-query expansion is that a paraphrased question can drift from the original question's precise meaning, and results from a drifted paraphrase are merged in alongside results from the original question, diluting an already reasonable ranking.

Different metrics disagreeing about which strategy is best

Multi-query expansion scored worse than plain vector search on precision and recall, but scored better on mean reciprocal rank in one comparison. This is not a contradiction, since mean reciprocal rank only measures how early the first relevant result appears, while precision and recall measure the quality of the entire set of results returned. Since every retrieved chunk, not only the first one, is passed to the generation stage together, precision and recall were judged the more relevant metrics for this system, and plain vector search was preferred despite its lower mean reciprocal rank in that comparison.

Multi-query's dependency on a working language model

Every retrieval method except multi-query only needs the vector store and an embedding model to run. Multi-query needs a working language model before any retrieval can even begin, since it must first generate paraphrased questions. This makes it the only retrieval method that can be entirely unavailable while every other method continues to work, and this had to be handled explicitly, with a clear skip message rather than a crash, whenever no language model is reachable.

7.3 Results on the real corpus

Retrieval strategies were compared using two separate labeled question sets built from real questions against the real corpus, containing 104 and 51 questions respectively.

Strategy	Precision	Recall	MRR	Latency
vector (104 questions)	0.308	0.63	0.412	16.7 ms
hybrid (104 questions)	0.136	0.274	0.304	1,665.7 ms
router (104 questions)	0.308	0.63	0.412	14.5 ms
multi_query (104 questions)	0.211	0.446	0.373	4,477.8 ms
vector (51 questions)	0.265	0.537	0.339	19.6 ms
hybrid (51 questions)	0.106	0.212	0.31	1,710.2 ms
router (51 questions)	0.265	0.537	0.339	13.4 ms
multi_query (51 questions)	0.235	0.503	0.51	4,404.7 ms

Table 7.1. Retrieval strategy comparison across two independent real question sets.

Plain vector search produced the best or tied best precision and recall in both question sets, while also being two to three orders of magnitude faster than hybrid search and multi-query expansion. The router method produced results identical to plain vector search in both runs, which reflects that this corpus's real questions are almost entirely ordinary conceptual questions rather than the date filtered or exact phrase questions the router's other routes are built for. Vector search was therefore selected as the retrieval method used for the production system, with the router available as an equivalent alternative.

8. Generation

The generation stage turns the retrieved chunks and the original question into a final answer. The prompt sent to the language model wraps the retrieved context in clear delimiters with an explicit instruction that this content is reference data rather than instructions, which is also the main defense against prompt injection described in Section 9. The model is asked to cite the source filename and page number for the information it uses.

8.1 Backends and models implemented

Backend	Models
Ollama	llama3.2 (general purpose, small footprint), mistral (strongest reasoning of the three, slowest), phi3 (fast, small)
Hugging Face	Qwen2.5-1.5B-Instruct (small, runs on CPU), Phi-3-mini-4k-instruct

A separate multimodal generator hands a retrieved image directly to a vision language model instead of relying on a stored text caption, which is discussed further in Section 11.

8.2 Results on the real corpus

Five candidate generation models were compared using the retrieved chunks from the winning vector search strategy, over 51 real labeled questions.

Model	Avg latency (s)	Avg faithfulness
llama3.2	32.2	0.357
mistral	92.19	0.381
phi3	120.37	0.356
Qwen2.5-1.5B-Instruct	pending	pending
Phi-3-mini-4k-instruct	pending	pending

Table 8.1. Generation model comparison on 51 real labeled questions. The two Hugging Face results were not yet completed at the time of writing.

Of the three completed results, phi3 was both the slowest and no more faithful than llama3.2, a result that contradicts phi3's own documented expectation of being fast and strong for its size on this system. mistral produced the highest faithfulness score but took nearly three times as long as llama3.2 for a gain of under seven percent, a lopsided tradeoff for a system where response time matters. llama3.2 delivered the best faithfulness per second of any candidate measured and is the leading candidate for the production system, subject to confirmation once the two pending Hugging Face results are available. The faithfulness score used here is a plain keyword overlap check rather than a semantic judgment, so a small gap between models should be read as a modest signal rather than a dramatic quality difference.

9. Safety

Documents ingested into the system are treated as untrusted content, since a source file could contain sensitive personal data or text specifically designed to manipulate the language model once it is pulled into the generation prompt. Two modules address this.

Module	Description
pii_redaction.py	Uses regular expressions to remove emails, phone numbers, and patterns resembling national identification numbers or credit card numbers before storage. This method catches structured personal data but not unstructured personal data such as a name written in ordinary prose. A named entity recognition based upgrade path using Microsoft Presidio is documented for that case.
prompt_injection.py	Provides two layers of defense. The first flags chunks containing known injection patterns at ingestion time for review, without automatically rejecting them, since legitimate text can resemble an injection attempt. The second, and more important, layer wraps all retrieved context defensively at generation time regardless of whether anything was flagged.

Both modules rely on pattern matching only, with no external models or paid services involved, which was a deliberate choice to keep this stage dependency free.

10. Evaluation

The evaluation stage supplies real, labeled ground truth so that every other measurement in this report has something honest to compare against. Without labeled data, claims such as the system works are simply assertions rather than measurements.

10.1 Tools implemented

Tool	Description
metrics.py	Implements precision at k, recall at k, mean reciprocal rank, and a keyword overlap faithfulness heuristic that checks whether a generated answer's wording actually comes from the retrieved context.
build_eval_set.py	Builds a labeled set of questions and their relevant chunk identifiers. Supports an interactive mode, where a person types a real question and marks which retrieved chunks are relevant by hand, and an auto label mode, where real, human written questions from a text file are labeled automatically by a local language model acting as a judge. A separate command removes duplicate questions, keeping whichever labeled version has more relevant chunks recorded.
ragas_eval.py	Uses the RAGAS library, configured to use a local Ollama model as judge instead of RAGAS's default paid service, to score the full retrieval and generation pipeline together on faithfulness, answer relevancy, context precision, and context recall. This is a more detailed evaluation than the metrics above but has not yet been exercised with real project data.

10.2 Difficulties encountered

The evaluation tool pointing at an empty collection

By default, the evaluation tool reads from one fixed vector store collection, while the comparison workflow described in Section 2.3 deliberately creates a separate, uniquely named collection for each combination of chunking method and embedding model under test. The first time this situation arose in practice, the evaluation tool was about to query a collection that had never been populated, which would have silently returned no results for every question. This was corrected by adding an option to target a specific collection directly.

A design tension in automating labeling

The evaluation tool was originally built specifically to avoid generating labels automatically, on the reasoning that a model's idea of relevance does not necessarily match a real user's judgment. Adding automated labeling required resolving this concern directly. The resolution kept the part of the original concern that mattered most: the questions themselves still have to be written by a real person, and only the relevance judgment for already retrieved passages is automated. This is a narrower and more defensible use of a model than generating an entire evaluation set from nothing.

Getting a reliable, parseable response from a local judge model

Asked to respond with a simple comma separated list of relevant passage numbers, a real language model does not always follow that format exactly. Reliable parsing required extracting digits from anywhere in the response rather than assuming an exact format, along with a specific check to distinguish a genuine no relevant results answer from a false match on the word none appearing elsewhere in the response. An early test of this parsing logic appeared to fail, which turned out to be a flaw in the test itself rather than in the code being tested, since the fake test model was reading the entire prompt, retrieved passages included, instead of only the actual question.

A high rate of no relevant chunks in the first large automated run

Running one hundred real questions through the automated labeling process produced no relevant chunks for roughly twenty nine percent of them. Investigating this rate revealed three distinct causes mixed together. Some cases were a genuine architectural limitation, since questions asking about a specific figure or diagram cannot be answered by a system that only searches text, given that images exist in a separate part of the system. Some cases were likely genuine retrieval misses, clustering specifically around how to and why questions rather than what is questions. Some cases were plausibly the judge model being more conservative than a human labeler would be. Distinguishing these causes with confidence requires spot checking a sample of the automated labels by hand.

Duplicate questions accumulating across repeated runs

Neither the interactive nor the automated labeling mode checks whether a question already exists in the file before adding a new entry, so running either mode more than once on the same file, which is the normal way to use this tool over time, silently produces duplicate entries for the same question. In the case that occurred during this project, four questions originally mislabeled by hand as having no relevant results were sitting alongside their own correctly labeled automated versions. Every metric that averages results per question counts both entries without knowing they are duplicates. This was corrected with a dedicated cleanup command that groups entries by question text and keeps whichever labeled version has more relevant chunks recorded.

Reported question count not matching the number of questions actually scored

Precision, recall, and mean reciprocal rank are only computed for a question if it has at least one labeled relevant chunk, since scoring a genuinely unanswerable question as a zero would unfairly penalize retrieval for correctly finding nothing. The total question count reported alongside these averages, however, includes every question in the file, including ones with no relevant chunks recorded. In one real run, a reported count of 104 questions corresponded to only 71 questions that actually contributed to the score, with the remaining 33 correctly excluded rather than counted as failures.

The same configuration scoring differently across two separate evaluation runs

The best performing retrieval configuration produced a noticeably different precision score on a 104 question labeled set compared to a separate 51 question set built afterward, which

demonstrates that a single evaluation run's exact numbers can shift meaningfully depending on which specific questions happen to be included, particularly at smaller sample sizes. What remained consistent across both runs was the relative ordering between different retrieval strategies, not the exact score achieved by any one of them. This is the practical principle this evaluation approach is built around: the comparison between approaches on the same evaluation set is more trustworthy than any single absolute number, until the evaluation set is large enough for that number to stop shifting meaningfully between runs.

11. Additional Exploration

Four further folders were implemented and independently benchmarked, extending beyond the required parts of the system. None of these are wired into the main question answering flow used day to day; These will be used for future improvement on this project, but were scrapped to respect the deadline.

11.1 Small language models

A model registry file records candidate small generation models as plain metadata, with no model loading involved, so tradeoffs in size, context length, and backend can be reasoned about before downloading anything. A companion benchmark script compares candidates on latency, memory footprint, and the faithfulness heuristic, reusing the same generator classes described in Section 8 rather than separate code paths.

11.2 Quantization

Two different mechanisms for reducing a model's memory footprint were implemented for comparison. The BitsAndBytes method loads a model in four bit or eight bit precision on the Hugging Face backend, but requires a CUDA capable graphics processor to provide any real benefit; it loads correctly on a CPU only machine but gives no improvement there. The GGUF method compares Ollama's pre quantized tag levels, such as four bit, five bit, and eight bit variants, against each other, and works well on a CPU without requiring a graphics processor.

11.3 Vision language models

Two backends for reasoning directly over a retrieved image, rather than a stored caption, were implemented. The Ollama backend offers llava, a strong general purpose model requiring a capable graphics processor or considerable patience on a CPU, and moondream, a much smaller model built for CPU and edge use. The Hugging Face backend offers moondream2, small enough to run reasonably on a CPU, and Qwen2-VL-2B-Instruct as a stronger but slower alternative.

11.4 Serving infrastructure

Two vLLM based serving methods were implemented to compare throughput under concurrent load against the simpler Ollama and Hugging Face serving already used elsewhere in the project. Both require a CUDA capable graphics processor, since vLLM's core performance

techniques are GPU only, making this the one part of the entire project that cannot run on a CPU only machine.

12. Utilities

A small set of supporting modules is used quietly throughout the rest of the system rather than invoked directly.

Module	Purpose
logging_config.py	Provides structured logging used across every other module.
retry.py	Applies exponential backoff specifically around calls to the local Ollama server, since a local server can fail transiently, for example while still loading a model.
tracing.py	Provides an optional integration with Arize Phoenix, a locally run trace visualization tool with no cloud account required, useful for debugging a chain of retrieval and generation steps.

13. Running the System

Section 2 introduced the four tools used to run this pipeline. This section summarizes the intended workflow for using them together.

1. Use `load_step.py` to run every applicable method at each stage and record comparison metrics, without affecting the production system.
2. Review the resulting numbers and select a configuration for chunking method, embedding model, vector store, retrieval strategy, and generation model.
3. Re-run that exact configuration through `stages.py` or `pipeline.py`, which write to the fixed `rag_chunks` collection used by the live system.
4. Use `api.py` to expose the committed configuration over a REST interface, or use `pipeline.py` directly from the command line.

This separation exists so that experimentation never risks the production index. A chosen configuration only becomes the live system once it has been deliberately re-run through the production tools; nothing transfers automatically between the two groups of tools.

14. Evaluation Metrics Explained

Each stage of the pipeline is measured differently, since good means something different at each one. Chunk shape is not retrieval quality, and retrieval quality is not generation quality. This section explains every metric reported at each stage.

14.1 Ingestion

The ingestion stage reports the total number of documents produced and a breakdown by type, specifically PDF text pages, images, and plain text files. For PDFs, this counts pages rather than files. These are accounting figures rather than quality measurements; correctness at this stage is checked by inspecting the resulting chunk statistics for anomalies, as described in Section 4.

14.2 Chunking

Chunk count, average word count, minimum and maximum word count, and standard deviation are reported for each method. A maximum value far larger than the average is the strongest signal that something went wrong upstream, and is exactly how the `structure_aware` extraction outlier described in Section 4 was found. The coefficient of variation, calculated as the standard deviation divided by the average, is a useful derived value for comparing methods that produce very different average chunk sizes; a lower value indicates more consistent chunk sizes. Processing time in seconds is also recorded, and was found to vary meaningfully between repeated runs of the semantic method.

14.3 Embedding

Vector dimensionality, total chunks embedded, total processing time, and time per chunk are reported for each candidate model. Before any real corpus text is processed, each model is checked against one paraphrase pair and one unrelated pair, producing two similarity scores and a boolean indicating whether the paraphrase score exceeds the unrelated score. The gap between these two scores, though not printed directly, is a useful derived measure of how cleanly a model separates similar from different meaning.

14.4 Storage

Total vectors stored, total time to write all vectors, average time per query, and self match accuracy are reported for each vector store backend. Self match accuracy checks whether querying with a vector that is already stored correctly returns that same chunk as the top result, and should equal one hundred percent; this is the measurement that would reveal a batching error that misaligned identifiers with vectors.

14.5 Retrieval

Precision at k measures, of the chunks retrieved, what fraction are actually relevant according to the labeled evaluation set. Recall at k measures, of all the relevant chunks that exist for a

question, what fraction were captured within the retrieved set. Mean reciprocal rank measures how close to the top of the results the first genuinely relevant result appeared, averaged across all questions, and only considers the first relevant hit rather than the full result set, which is why a strategy can score well on mean reciprocal rank while scoring poorly on precision and recall, as observed with multi-query expansion in Section 7. Average latency per question is also recorded. The reported number of questions reflects the size of the evaluation set rather than the number of questions that actually contributed to the precision, recall, and mean reciprocal rank averages, since a question with no labeled relevant chunks is correctly excluded from those averages rather than counted as a failure.

14.6 Generation

Average latency per generated answer and average faithfulness are reported for each candidate model. Faithfulness is measured with a plain keyword overlap check between the generated answer and the retrieved context, rather than a semantic judgment, and is useful for catching an answer that is clearly unsupported by the retrieved context, but a small difference between two models should be read as a modest signal rather than a large quality gap. The number of answers completed is also recorded, which matters if a comparison run is interrupted partway through.

14.7 A more detailed alternative

Where the metrics above mostly score one stage in isolation, the RAGAS based evaluation described in Section 10 scores the full retrieval and generation pipeline together, judged by a local language model rather than a keyword overlap formula, and is available in this project but has not yet been exercised with real project data.

15. Conclusion

This project implemented and evaluated a complete local retrieval-augmented generation pipeline across all six stages: ingestion, chunking, embedding, storage, retrieval, and generation. Every stage was implemented with multiple interchangeable methods, and a dedicated comparison tool was built so that a configuration could be selected using measured evidence from the real document collection rather than assumption.

The evidence gathered points to a consistent conclusion across several stages: the simpler method frequently outperformed the more elaborate one on this particular corpus. Plain vector search beat both hybrid search and multi-query expansion on precision and recall while being dramatically faster. A smaller, cheaper embedding model separated meaning more cleanly than larger, more expensive alternatives. This is a genuinely useful finding, not merely a default choice; it demonstrates the value of measuring rather than assuming which technique is best suited to a given corpus and question set.

The project also surfaced several concrete engineering issues that were identified and corrected during development, including missing source metadata across every chunking method, a

vector store batch size limit that only appeared at real corpus scale, and a collection mismatch in the evaluation tooling. Each of these was found through direct testing against real or realistic data rather than through code review alone, which reinforces the value of end to end testing over inspection.

Several limitations remain and are stated here directly rather than omitted. The `structure_aware` chunking method contains an unresolved extraction bug affecting at least one page in the real corpus. Image context extraction, described in early project notes, could not be confirmed as implemented in the delivered code. The RAGAS based full pipeline evaluation has not yet been exercised with real project data. Two of the five candidate generation models had not completed evaluation at the time of writing. Multimodal image retrieval using CLIP has not yet been incorporated into the production configuration, since the embedding model selected for text, `all-MiniLM-L6-v2`, cannot embed images, and CLIP's own suitability for this corpus's long form text retrieval has not yet been measured against the selected text embedding model.

The current production configuration selected on the basis of this evaluation uses `parent_child` chunking, the `all-MiniLM-L6-v2` embedding model, the Chroma vector store, and plain vector search for retrieval, with `llama3.2` as the leading generation model candidate pending completion of the remaining comparisons. This configuration is intended to be re-run through the production tools described in Section 13 to build the live system evaluated in this report.

Appendix A: Development History

This appendix records the development of the project in chronological order, organized into phases.

Phase 1: Initial review and first round of fixes

The codebase was audited against every stated requirement. A required REST API was found to be entirely missing and was built from scratch, exposing ingestion, query, document listing, and health check endpoints while reusing the existing pipeline modules rather than duplicating logic. While testing the new API end to end, three pre-existing defects were identified and corrected: missing source metadata across all six chunking methods, a hybrid retrieval function that never returned metadata at all, and a hybrid retrieval option that was fully coded but unreachable from the command line interface, along with a related crash in the query router's keyword heavy route.

Phase 2: Extending method comparison to new code

The retrieval benchmark tool, which originally compared only three of five working strategies, was extended to cover all five. A query comparison endpoint was added to the REST API so multiple retrieval strategies could be compared side by side over an HTTP request. An inefficiency discovered during this work, where the reranker model was reloaded from scratch on every request, was corrected by caching a single loaded instance.

Phase 3: Building the checkpointed comparison tool

A tool was built to run each of the six pipeline stages independently, with every stage reading its input exclusively from a file on disk rather than from an in memory variable, and every stage running every applicable method rather than one. This was verified in two ways: a complete run against a real test corpus, and a separate test proving that a later stage succeeds correctly in a brand new process using only what earlier stages had written to disk, confirming that stage independence was a real guarantee rather than only a design intention.

Phase 4: Bugs found only once real data was used

The vector store's write function was found to send an entire batch to Chroma in a single call, which failed once the real corpus's parent_child chunk count exceeded Chroma's internal row limit for a single call. This was corrected by batching writes automatically using the limit reported by Chroma's own client, with the same fix applied proactively to the Qdrant backend. Separately, the evaluation tool was found to always read from one fixed default vector store collection with no way to target a specific comparison collection, which was corrected by adding a collection override option.

Phase 5: Automated relevance labeling

An automated labeling mode was added to the evaluation tool, allowing real, human written questions from a text file to be labeled by a local language model acting as a judge, while keeping the requirement that questions themselves still be written by a person. A parser was built to handle imperfect real model output reliably, and was verified after an early test that appeared to show a defect turned out to be a flaw in the test itself rather than in the code being tested.

Phase 6: Evaluation set hygiene

Neither labeling mode checked for an existing entry with the same question before appending a new one, which allowed duplicate entries to accumulate across repeated use, sometimes with one stale entry and one correct entry for the same question. A deduplication command was added that keeps whichever labeled version of a duplicate question has more relevant chunks recorded.

Phase 7: Documentation

Reference material was written for the ingestion, chunking, embedding, retrieval, and evaluation stages, each grounded in the real defects found and real numbers generated during the phases above, followed by a complete project reference covering every folder, the justification for the ingestion method, and an explanation of every evaluation metric used. This appendix and the rest of this report represent the final stage of that documentation process.

Phase 8: Activating an unused prompt injection defense

A function in the prompt injection safety module, intended to wrap retrieved context in explicit delimiters marking it as data rather than instructions, existed but was never called from anywhere in the codebase; the prompt actually sent to every generator concatenated raw chunk text with no such protection. The retrieval-augmented generation prompt builder was corrected to call this function on the assembled context before it reaches the language model.

Phase 9: Incremental ingestion silently duplicating image vectors

The command line ingestion tool's incremental mode was found to register only text chunk identifiers in its manifest, never image chunk identifiers; because each image's identifier was randomly generated rather than derived from its content, every incremental run treated every image as new and re-stored it, silently duplicating image vectors on every run. The fix collects text and image chunk identifiers into one shared per-file record written to the manifest together, rather than handling images as an afterthought that could overwrite or be overwritten by the text record for the same file, since a PDF's page text and its embedded images share the same source filename. Verified by running incremental ingestion three times over an unchanged folder with a fake in-memory store: before the fix, the stored count grew on every run, two, then three, then four; after the fix, it remained constant across all three runs.

Phase 10: A first approach to multimodal retrieval, through captioned images and CLIP's text tower

A first design for retrieving images alongside text was built around a small vision language model writing a short caption for each image, with the caption then embedded through CLIP's own text tower rather than through the pipeline's primary text embedder, on the reasoning that a short caption fits CLIP's roughly seventy seven token text limit far better than long prose, and that text embedded through CLIP and images embedded through a separate model such as all-MiniLM cannot occupy the same vector space. This required a second vector store collection for images and a fusion step, using reciprocal rank fusion, the same technique already used elsewhere for hybrid and multi-query retrieval, to combine the separate image collection's ranked results into whichever text retrieval strategy was already in use; because fusion operates on rank rather than raw score, it worked with any text strategy's output unmodified. The command line pipeline and the checkpointed six stage tool were both given a new image embedder option, left unset by default so existing behavior was unaffected, and the incremental manifest's per-file record gained a second, parallel list of image chunk identifiers so that text and image vectors, now potentially split across two different stores, could be cleaned up independently when a file changed or was deleted. The multimodal generator was also updated so that when more than one image is retrieved, images not sent directly to the vision model still surface their actual caption in the text context rather than only their filename. Verified with a full ingest, store, and query cycle using fakes standing in for CLIP, the vision language model, and the vector stores, confirming that captions were stored as real searchable text, that incremental mode remained stable across the new second collection, and that existing behavior was unaffected when the new option was left unset.

Phase 11: A circuit breaker for a silently broken captioning backend

The first run of the captioning approach above against the real production corpus saw every one of more than fifteen hundred images fail identically, each with a disconnection error from the vision language model backend; because each failure was caught and handled individually, the run continued regardless, printing the same failure over a thousand times and producing a checkpoint full of empty, useless captions rather than reporting that the run was fundamentally broken. The captioning function was corrected to track consecutive failures and raise immediately once three occurred in a row, so that an isolated failure elsewhere in an otherwise successful run does not trip it, but a systemically broken backend is caught within the first few images rather than after an entire corpus has already been wasted on it.

Phase 12: Import time cost mistaken for a runtime problem

Both the command line pipeline and the checkpointed six stage tool imported all six chunking methods unconditionally at the top of the file, including in code paths, such as storing vectors, generating an answer, checking status, or clearing checkpoints, that never chunk anything at all; one of those chunking modules depends on a text splitting library that itself depends on a large transformer library, which turned out to be the actual cause of multi minute hangs previously suspected to come from the language model backend. The imports were moved to be local to only the specific functions that actually chunk, matching the lazy import pattern already used elsewhere in both files for retrieval strategies. Verified that neither the text

splitting library nor the transformer library loads merely from importing either file, and re-ran a complete six stage cycle, including parent-child chunking, to confirm nothing had broken.

Phase 13: A system proxy silently discarding image requests to a local model server

Every captioning request to the local vision language model server failed identically while plain text requests to the same server succeeded without issue, which traced to the request being routed through an HTTP proxy handler rather than reaching the server directly; a system level proxy configuration was intercepting local traffic and discarding the larger, image bearing requests specifically. Disabling the client's use of environment based proxy configuration resolved the failure, confirmed not by assumption but by reading both the client library's and its underlying HTTP library's source directly to verify that this single setting governs all automatic proxy detection. The same client construction pattern existed in the text generation, text embedding, and evaluation judge model clients, and the same correction was applied to each.

Phase 14: A second, independent branch approach to multimodal retrieval

An alternative design for retrieving images alongside text was later built and adopted alongside the first: text chunks remained on the pipeline's existing dedicated text embedder, while images were embedded through CLIP into an entirely separate collection, each also receiving a caption from a vision language model that was embedded through the same text embedder used for ordinary text chunks and stored alongside them, making an image reachable by plain semantic search as well as by CLIP's own cross modal similarity. At answer time the text and image branches were drafted independently by the language model and the vision language model respectively, with either branch dropped from the final answer if its retrieval score fell below a threshold, or if the model handling that branch judged, through a strict required response it must produce when it genuinely cannot answer, that its retrieved content was not relevant; a final language model call combined both drafts only when both were judged usable, and neither branch surviving produced a fixed message reporting insufficient information with no further language model call, so that nothing was invented at that final stage either. This was wired into the command line pipeline, the checkpointed six stage tool, and the REST API, with the image embedding and captioning step and the image branch retrieval step each implemented once and shared across all three interfaces, so that a defect found and fixed in one could not silently persist in another.

Phase 15: Defects found while building the second approach

Two further defects surfaced while wiring the second multimodal design through all three interfaces. A caption chunk's own constructor treated two of its arguments as structured fields on the resulting object rather than as entries in its metadata, so building a caption chunk the most direct way silently lost the link back to its source image that made the caption useful in the first place; this was corrected by setting that metadata explicitly after construction, in the single function responsible for building every caption chunk across all three interfaces.

Separately, and independently of the image vector duplication already described in an earlier phase, the incremental manifest was found to group and match chunks by a document's bare filename rather than its full path; two files sharing a filename in different subfolders collided on that key, so a manifest entry was written for only one of them while the other was silently treated as new and reprocessed on every subsequent incremental run, indefinitely. This was corrected by keying the manifest to each document's own internal identifier, already reliably preserved onto every one of its chunks by every chunking method including parent-child, mapped back to its true file path, rather than reconstructing that path by searching for a filename match.

Phase 16: Identifiers that did not survive re-ingestion

Both documents and their chunks were identified by a randomly generated value unrelated to their content, so re-ingesting an unchanged file, whether by re-running the pipeline or by re-uploading the same file through the REST API, inserted a fresh duplicate copy under new identifiers every time rather than updating anything already stored; from the outside, this looked like the same ingestion running repeatedly without ever making progress. The fix derives both identifiers from the content itself rather than from the file's path on disk, since the REST API stages every upload under a freshly randomized temporary filename specifically to avoid collisions between simultaneous uploads, which would have defeated a path-based fix for exactly the case it needed to address; images derive their identifier from their own bytes for the same reason. Verified that re-uploading an unchanged file three times converged to a single stored entry rather than three, that genuinely different content still received its own separate entry, and that a complete six stage pipeline run repeated over unchanged data no longer duplicated the store either.

Phase 17: Repeated model loading inside a long running server

The image embedding and captioning functions shared with the command line tool always loaded their own copy of the CLIP model and the vision language model internally; this was inexpensive for the command line tool, where each stage is a short lived process that only calls them once, but meant the REST API, a long running process, reloaded both models from scratch on every single request, which was the actual cause of the API feeling slow in exactly the cases where the equivalent command line run felt fast. The functions were corrected to optionally accept an already loaded model and store, defaulting to the previous behavior when none is supplied, and the API was corrected to load and cache a single instance of each the same way its language model and reranking components were already cached. Verified that three separate requests, each with a different image, loaded the relevant models once between them rather than three times.

Phase 18: A metadata value the vector store's own validation rejected

Enabling personal information redaction alongside content that genuinely contained an email address and a phone number crashed storage outright, because the redaction step recorded how many matches of each category were found as a nested structure, and the vector store's own

metadata validation accepts only plain strings, numbers, booleans, lists, or nothing at all, rejecting anything nested. This was corrected by recording the same information as a text encoded structure instead, which every existing caller could still interpret correctly, since each only checked whether anything had been redacted at all rather than reading the structure's contents. Confirmed directly against the vector store's own validation function rather than assumed, and again end to end against a live request containing genuine matching content.

Phase 19: A single unreadable image ending an otherwise successful run

Source documents occasionally embed images in a format that the extraction library reconstructs as an incomplete or malformed byte stream, which the underlying image library then refuses to decode; because the image embedding step processed an entire batch of images in one call with no handling for an individual failure, one such image anywhere in a batch ended the whole request, discarding even the text content from the same document that had already been successfully embedded and stored earlier in that same call. This was corrected by handling each image's failure individually and substituting an all zero vector for anything that could not be decoded, which is inert at retrieval time, since its similarity to any query is always zero, rather than allowing one unreadable image to end the entire batch.

Appendix B: Model and Method Reference Table

B.1 Embedding models

Model	Backend	Dimensions	Notes
all-MiniLM-L6-v2	Hugging Face	384	Selected for the production system.
bge-small-en-v1.5	Hugging Face	384	Fastest candidate tested.
bge-base-en-v1.5	Hugging Face	768	Higher cost, no measured quality gain on this corpus.
nomic-embed-text	Ollama	768	Slowest working candidate, no batching support.
mxbai-embed-large	Ollama	1024	Failed on the real corpus due to a context length limit.
ViT-B-32 (CLIP)	open_clip	512	Only model capable of embedding images; not yet compared for text retrieval quality.

B.2 Generation models

Model	Backend	Parameters	Notes
llama3.2	Ollama	3.0 billion	Leading candidate based on completed comparisons.
mistral	Ollama	7.0 billion	Highest faithfulness score, slowest of the completed candidates.
phi3	Ollama	3.8 billion	Slowest and no more faithful than llama3.2 on this system.
Qwen2.5-1.5B-Instruct	Hugging Face	1.5 billion	Evaluation pending at time of writing.
Phi-3-mini-4k-instruct	Hugging Face	3.8 billion	Evaluation pending at time of writing.
Qwen2.5-0.5B-Instruct	Hugging Face	0.5 billion	Registered as a latency floor baseline; not part of the main comparison.

B.3 Vision language models

Model	Backend	Notes
llava	Ollama	Strong general purpose model, requires a capable graphics processor for practical speed.
moondream	Ollama	Small, built for CPU and edge use.
moondream2	Hugging Face	Default choice for the Hugging Face backend, small enough for reasonable CPU speed.
Qwen2-VL-2B-Instruct	Hugging Face	Stronger and slower alternative.

B.4 Reranking model

Model	Purpose
cross-encoder/ms-marco-MiniLM-L-6-v2	Scores a question and a candidate chunk together for a more accurate, slower second pass after initial retrieval.

Appendix C: Production Deployment on the Real Corpus

After the comparison work described in the body of this report, the selected configuration, parent_child chunking with the all-MiniLM-L6-v2 embedding model and Chroma storage, was run through the production tools described in Section 13 against the full real corpus of ten source documents. This appendix records the results of that run and the current deployment status.

C.1 Per-file ingestion results

The following table shows, for each source PDF, how many full-page scan images were filtered out and how many pages required the OCR fallback because no native text layer was found.

Source file	Scans filtered	Pages needing OCR
Learn and Master Painting (Gayle Levee)	1	1
A Text-book of the History of Painting (Van Dyke)	385	1
Painting in the Dutch Golden Age (National Gallery of Art)	35	1
A Treatise on Landscape Painting	290	20
Handbook of Oil Painting	900	4
Historical Painting Techniques, Materials and Studio Practice (Berland)	89	3
Original Treatises on the Arts of Painting, Vol. 1 (Merrifield)	644	634
Original Treatises on the Arts of Painting, Vol. 2 (Merrifield)	610	594
The Practice and Science of Drawing	836	30
Treatise on Painting	745	9
Water Colour	672	41
Total	5,207	1,338

Table C.1. Per-file ingestion results on the real corpus.

Across the full corpus, 1,338 of the 3,668 total PDF text pages, or 36.5 percent, required the OCR fallback described in Section 3.4. This confirms directly, with real per-file evidence, the claim made in that section that some books in this corpus are almost entirely raw scans with no native text layer. The two Merrifield volumes are the clearest case: 634 of that volume's pages and 594 of the second volume's pages needed OCR, meaning the native text extraction path contributed almost nothing for these two books specifically, and the OCR fallback was not an edge case for them but the primary extraction method. In total, 5,207 full-page scan images were correctly filtered out across the corpus using the placement-based method described in Section 3.3.

Total ingestion output was 4,791 raw documents: 3,668 PDF text pages, 1,117 images, and 6 plain text files.

C.2 Chunking and embedding at production scale

Chunking with the `parent_child` method produced 16,431 total chunks: 15,314 text chunks and 1,117 image chunks. Embedding with the selected `all-MiniLM-L6-v2` model successfully processed all 15,314 text chunks. The 1,117 image chunks could not be embedded, since this model does not support images, and the system correctly reported this rather than failing silently, printing a message recommending the CLIP backend instead. This confirms, at full production scale, the finding discussed in Section 5: the embedding model selected for text cannot embed images, and the corpus's 1,117 images are not currently searchable in the production index. This remains a limitation to be addressed, either by adding a separate CLIP-based image collection or by re-evaluating CLIP's suitability for this corpus's text retrieval before switching the whole index to it, as discussed in Section 15.

C.3 Storage

The 15,314 text vectors were stored successfully in Chroma, which the system marked as the durable production index, meaning the intermediate checkpoint files from the earlier stages are no longer required for retrieval or generation to work. This step was attempted three times in total before completing successfully: the first attempt failed with the Chroma batch size error described in Section 6, since it was run before the automatic batching fix was applied; the second attempt succeeded for Chroma but could not reach Qdrant, since no Qdrant server was running at the time; the third attempt succeeded for both backends once a local Qdrant server had been started. Only Chroma was used for the final production index, consistent with the choice explained in Section 6.1.

C.4 Generation model completion status

A first attempt to compare all five candidate generation models did not complete. The Ollama model `phi3` failed on this attempt with an out-of-memory error while allocating its context buffer, requesting approximately 24 gigabytes for a model of only 3.8 billion parameters, which points to an oversized default context length rather than the model's size itself. The Hugging Face model `Qwen2.5-1.5B-Instruct` failed with an unspecified error after its weights had already loaded successfully. The Hugging Face model `Phi-3-mini-4k-instruct` began downloading its 3.09 gigabyte weight file but stalled at an impractical download speed and was interrupted manually.

A second attempt was run afterward. On this attempt, `llama3.2`, `mistral`, and `phi3` all completed successfully, with `phi3` succeeding this time rather than failing with an out-of-memory error as it had on the first attempt; these are the three results already reported in Table 8.1. `Qwen2.5-1.5B-Instruct` again failed with an unspecified error after its weights loaded successfully, this time on a fully downloaded and cached model rather than one being downloaded fresh, which suggests the failure is unrelated to the download itself. `Phi-3-mini-4k-instruct` this time required

downloading a larger, sharded 7.64 gigabyte version of its weights, and this download was again interrupted manually after not completing in a practical amount of time.

At the time of writing, both Hugging Face generation models remain incomplete for reasons distinct from each other: Qwen2.5-1.5B-Instruct fails after loading for a reason not yet diagnosed, while Phi-3-mini-4k-instruct has not yet finished downloading its weights on this machine's connection. These should be treated as open items rather than as results, and are marked accordingly in Table 8.1 and Appendix B.

C.5 Current deployment status

As of this writing, the ingestion, chunking, embedding, and storage checkpoints for the production index are complete, using the configuration selected in the body of this report. Retrieval and generation have not yet been run against the production index itself; the retrieval and generation results reported in Sections 7 and 8 come from the comparison collection built during the evaluation phase, not yet from this production collection. Running the production system's retrieve and generate stages against real questions is the immediate next step to complete the deployment.